

UNIDAD 2 - LA VPS: APROVISIONAR, ENTRAR Y CERRAR

Taller de aula: cerrar un servidor sin tener el servidor

Programacion 3 - Tecnicatura Universitaria en Programacion - UTN FRM

Modalidad	En grupos de dos o tres, en el aula. Se entrega antes de irse : lo que no se termina en la mesa no se termina.
Requisitos	No hace falta tener el VPS provisionado. Tampoco hace falta internet. Solo el apunte del capitulo 2 a mano.
Tiempo	70 minutos. Partes 1 a 5, mas la Parte 6 que se lleva al proyecto.
Puntaje	100 puntos: 1 (25) + 2 (15) + 3 (20) + 4 (15) + 5 (15) + 6 (10).
Que se evalua	Criterio operativo: decidir que se expone y que no, y poder justificar cada decision con el mecanismo que la explica.
Integrantes	

Antes de arrancar

Este taller no pregunta que recordas: pregunta que **decidirias**. Todas las salidas de comandos que vas a leer estan impresas aca; no hay nada que ejecutar. Cuando dudes, busca la seccion del apunte: encontrarla tambien es parte del ejercicio. Y una regla de oro para todo el taller: **una decision sin justificacion no suma puntaje**. "Porque si" no es una razon de ingenieria.

Parte 1 - Inventario de puertos y decision de exposicion

25 puntos

Un grupo de la comision levanto su proyecto en el VPS y corrio `ss -tlnp`. Esta es la salida completa, sin recortar. El proyecto tiene una API en FastAPI, una base MongoDB, un Redis de cache y el panel de Easypanel.

```
root@vps-tup:~# ss -tlnp
State  Recv-Q  Send-Q  Local Address:Port  Process
LISTEN  0        4096    0.0.0.0:22      users:(( "sshd",pid=791))
LISTEN  0        4096    0.0.0.0:80      users:(( "traefik",pid=1620))
LISTEN  0        4096    0.0.0.0:443     users:(( "traefik",pid=1620))
LISTEN  0        4096    0.0.0.0:3000    users:(( "docker-proxy",pid=1704))
LISTEN  0        4096    0.0.0.0:8000    users:(( "docker-proxy",pid=1893))
LISTEN  0        4096    0.0.0.0:27017   users:(( "docker-proxy",pid=1955))
LISTEN  0        511     127.0.0.1:6379   users:(( "redis-server",pid=1512))
```

1.1 Completa el inventario

14 puntos (2 por fila)

Una fila por puerto. En "Rango RFC 6335" escribi si es bien conocido, registrado o dinamico. En "Alcanzable desde internet" respondes lo que **deberia** ser, no lo que hoy es.

Puerto	Proceso	Interfaz en la que escucha	Rango RFC 6335	Debe ser alcanzable desde internet?	Por que (una linea)
22					
80					
443					
3000					
8000					
27017					
6379					

1.2 Elegi la estrategia de cierre*6 puntos (2 por servicio)*

Para los tres servicios que decidiste que **no** deben verse desde internet pero hoy escuchan en 0.0.0.0, elegi una de las estrategias de la seccion 2.8.3 y justificala. Ojo: "agregar una regla ufw deny" no es una de ellas, y en la Parte 2 vas a ver por que.

1.3 Predecí la evidencia externa*5 puntos*

Suponiendo que el grupo aplica exactamente las correcciones que propusiste, escribi la salida que deberia dar nmap -Pn tudominio.me corrido desde otra red. Una linea por puerto. Y decime por que esta comprobacion vale mas que cualquier otra del capitulo.

Parte 2 - El recorrido del paquete

15 puntos

Dos paquetes salen de la misma notebook, en la misma red, con un segundo de diferencia. Los dos llegan a la IP publica del mismo servidor. Uno lo filtra ufw y el otro no. No es magia ni un error de Ubuntu: es el recorrido.

```
Paquete A destino 203.0.113.45:22 -> sshd, que corre en el host
Paquete B destino 203.0.113.45:27017 -> mongo, que corre en un contenedor
publicado con -p 27017:27017
```

2.1 Traza los dos recorridos

8 puntos (4 por paquete)

Complete la secuencia de puntos de enganche de netfilter que atraviesa cada paquete, desde que entra por la placa de red hasta que llega al proceso. Marca con un circulo el momento exacto en que los dos caminos se separan.

	Paquete A (puerto 22, proceso en el host)	Paquete B (puerto 27017, contenedor publicado)
1		
2		
3		
4		
5		

2.2 Explicalo en una frase de ingenieria

4 puntos

Completa: "sudo ufw deny 27017 no bloquea nada porque ufw escribe sus reglas en la cadena _____, y el paquete publicado por Docker pasa por la cadena _____, de modo que _____."

2.3 Que comando muestra la verdad de adentro

3 puntos

Nombra el comando que permite leer el conjunto de reglas real del nucleo, con las cadenas que creo ufw y las que creo Docker una al lado de la otra. Y explica por que ese comando, aun siendo mas honesto que ufw status, sigue sin ser la verificacion definitiva.

Parte 3 - El guion de endurecimiento

20 puntos

Doce pasos, desordenados, sobre un servidor recién creado al que hoy se entra con contraseña de root. El objetivo es dejarlo con acceso solo por clave y con el firewall activo, **sin quedarse afuera en el intento**. Este ejercicio es el que más alumnos pierde en la práctica: el orden no es una preferencia de estilo, es la diferencia entre tener servidor y no tenerlo.

- a) `sudo ufw allow OpenSSH`
- b) `sudo ufw enable`
- c) `ssh-keygen -t ed25519 -C "matias@notebook"`
- d) editar `/etc/ssh/sshd_config` y poner `PasswordAuthentication no`
- e) `sudo ufw allow 80/tcp && sudo ufw allow 443/tcp`
- f) `ssh-copy-id root@203.0.113.45`
- g) `sudo systemctl restart ssh`
- h) abrir una SEGUNDA terminal y entrar por SSH, sin cerrar la primera
- i) `sudo apt update && sudo apt install fail2ban -y`
- j) `sudo ufw status verbose`
- k) `nmap -Pn tudominio.me` (desde otra máquina, en otra red)
- l) `ssh root@203.0.113.45` y comprobar que NO pide contraseña

3.1 Ordena los doce pasos

10 puntos

Escribi las doce letras en el orden correcto de ejecución.

3.2 Marca los tres puntos de no retorno

6 puntos (2 por punto)

Tres de esos pasos, ejecutados fuera de orden, te dejan afuera del servidor y solo se arreglan por la consola de emergencia de Hostinger. Identificalos y decime en cada caso que error concreto los convierte en trampa.

3.3 La red de seguridad

4 puntos

Dos de los doce pasos no cambian absolutamente nada en el servidor: solo miran. Identificalos y explica que principio de trabajo representan y por qué van exactamente donde los pusiste.

Parte 4 - Auditoria de un archivo de configuracion**15 puntos**

Un grupo entrego este fragmento de `/etc/ssh/sshd_config` como "servidor endurecido". Tiene **cinco problemas reales y una linea que parece un problema y no lo es**.

```
# /etc/ssh/sshd_config (fragmento entregado por un grupo)
1  Port 22
2  PermitRootLogin yes
3  PubkeyAuthentication yes
4  PasswordAuthentication yes
5  PermitEmptyPasswords yes
6  MaxAuthTries 30
7  X11Forwarding yes
8  AuthorizedKeysFile .ssh/authorized_keys
9  UsePAM yes
```

4.1 Los cinco problemas*10 puntos (2 por problema)*

Para cada uno: numero de linea, valor corregido y una linea de justificacion. Sin justificacion no suma.

4.2 La linea que no es un problema*5 puntos*

Un integrante propone cambiar la linea 1 a `Port 2222` "para que no lo encuentren los bots". Identifica por que esa medida **no** mejora la seguridad del servidor en terminos del capitulo, y nombra ademas el riesgo operativo concreto que introduce si se aplica en el orden equivocado.

Parte 5 - Peritaje: reconstruir un incidente**15 puntos**

Un grupo pide ayuda: "nos borraron una tabla de la base y no entendemos como, si el SSH estaba blindado". Esta es toda la evidencia que trajeron. Leela entera antes de escribir nada.

```
root@vps-tup:~# docker ps --format "{{.Names}}\t{{.Ports}}"
tup-api      0.0.0.0:8000->8000/tcp
tup-db       0.0.0.0:5432->5432/tcp
tup-front    0.0.0.0:8080->80/tcp

root@vps-tup:~# sudo ufw status
Estado: activo
Hasta      Accion      Desde
22/tcp     ALLOW IN    Anywhere
80/tcp     ALLOW IN    Anywhere
443/tcp    ALLOW IN    Anywhere
5432/tcp   DENY IN     Anywhere

--- desde la notebook de un companero, en otra red ---
$ nmap -Pn 203.0.113.45
PORT      STATE  SERVICE
22/tcp    open   ssh
80/tcp    open   http
443/tcp   open   https
5432/tcp  open   postgresql
8000/tcp  open   http-alt
8080/tcp  open   http-proxy

root@vps-tup:~# grep -c "Failed password" /var/log/auth.log
0

--- registro del contenedor de la base ---
2026-08-14 03:12:07 UTC [4412] FATAL: password authentication failed for user "postgres"
2026-08-14 03:12:09 UTC [4415] FATAL: password authentication failed for user "postgres"
2026-08-14 03:12:11 UTC [4419] FATAL: password authentication failed for user "postgres"
2026-08-14 03:14:41 UTC [4602] LOG: connection authorized: user=postgres database=tup
2026-08-14 03:19:55 UTC [4602] LOG: statement: DROP TABLE operaciones;
```

5.1 Reconstrui la cadena de hechos**5 puntos**

En cinco lineas como maximo, contame que paso, en orden, desde que el grupo levanto el proyecto hasta el DROP TABLE. Sostene cada afirmacion en una linea concreta de la evidencia.

5.2 Las dos contradicciones aparentes*4 puntos (2 cada una)*

La evidencia parece contradecirse dos veces. Explica cada caso: (i) ufw status dice que el 5432 esta bloqueado y nmap lo ve abierto; (ii) hubo una intrusion y sin embargo auth.log no registra un solo intento fallido.

5.3 Los principios violados*3 puntos*

De los tres principios de la seccion 2.12.1, indica cuales se violaron y en que decision concreta del grupo.

5.4 Las tres correcciones, en orden*3 puntos*

Escribi las tres primeras medidas que aplicarias, en el orden en que las aplicarias. Justifica el orden, no solo el contenido.

Parte 6 - El plan de puertos de tu propio proyecto**10 puntos**

Esta parte no es un ejercicio de laboratorio: es el plano de lo que vas a desplegar en las Clases 4 y 5. El dominio ya lo tenes de la Clase 1; el servidor puede estar o no estar, no importa. Lo que define ahora es que se publica y que no, **antes** de escribir un solo `docker run`.

Servicio	Subdominio que va a usar	Puerto interno	Se publica al mundo?	Como llega el trafico	Registro DNS necesario
Panel de administracion					
Frontend					
API					
Base de datos					

6.1 El registro que te ahorra trabajo**5 puntos**

De todos los registros DNS de tu tabla, cuantos tuviste que crear a mano en la Clase 1 y cuantos te resuelve un solo registro? Nombralo y explica que problema evita cuando en la Clase 5 agregues un servicio nuevo.

6.2 La linea que te vas a agradecer**5 puntos**

Mira tu tabla y responde: si mañana alguien de tu grupo levanta la base "un ratito con `-p 5432:5432` para probar con DBeaver desde su casa", que le decis? Escribi la respuesta tecnica **y la alternativa concreta** que le ofreces, porque la necesidad es legitima.

Cierre - Para discutir en la mesa**sin puntaje**

La conclusion del capitulo es una sola frase: **el puerto mas seguro es el que nunca se publico**. No el que esta bloqueado, no el que tiene contrasena fuerte, no el que esta en un numero raro. El que no existe hacia afuera.

Discutan cinco minutos, sin escribir: en el codigo que ya escribieron en esta carrera, donde estan sus "puertos publicados de mas"? Un endpoint que devuelve mas campos de los que la pantalla usa. Un usuario de base con permisos de administrador porque era mas rapido. Un repositorio publico con un archivo `.env` adentro. Una variable de entorno que llego al bundle del frontend. En cada uno de esos casos, cual seria el equivalente a correr `nmap` desde afuera?

Lo que se entrega

Las hojas del taller, con el nombre de los tres integrantes en la primera. Si la Parte 6 les quedo a medias, entreguenla igual como esta: esa tabla es el plano del despliegue de las Clases 4 y 5, y la vamos a retomar. Lo que **no** se entrega en blanco es la justificacion: prefiero una decision discutible bien fundamentada que una correcta sin razon.